

Chapter 15: Routing

1. Overview

The Angular Router is the mechanism that maps URLs to application state — specifically, to a tree of activated components. It turns a single-page application into something that *feels* like a multi-page application: the browser's back/forward buttons work, URLs are bookmarkable and shareable, deep links load the right view, and the app can be split into lazily-loaded chunks that download only when needed.

At its core, routing solves one problem: **given a URL, decide which components to render, in which layout slots (outlets), with which data**. Everything else — guards, resolvers, preloading, named outlets, matrix params — is built around that core matching-and-activation loop.

Interviewers probe routing heavily because it touches almost every architectural concern in a real app: code-splitting (lazy loading), security (guards), data-fetching timing (resolvers), performance (preloading strategies), and UX (scroll restoration, transition state). A candidate who can explain *how* the router builds its internal state tree and *why* certain lifecycle hooks fire in a certain order stands out from one who has only used `routerLink` in templates.

This chapter covers the modern **standalone, function-based router API** (`provideRouter`, `withComponentInputBinding`, functional guards/resolvers) introduced in Angular 14–15 and now the default in Angular 17+, while also noting the equivalent `RouterModule.forRoot()` NgModule-based API since many production codebases and interview questions still reference it.

2. Core Concepts

2.1 Router Setup

Two setup styles exist:

Standalone (modern, Angular 14+ `bootstrapApplication` / Angular 17+ default):

```
bootstrapApplication(AppComponent, {
  providers: [provideRouter(routes)]
});
```

NgModule-based (classic):

```
@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule {}
```

`forRoot()` is called exactly once, at the application root, and configures the singleton `Router` service plus `Location` strategy. Feature modules use `RouterModule.forChild(routes)` — it registers routes without re-providing the router service (which must remain a single app-wide singleton).

`provideRouter()` is the standalone equivalent of `forRoot()`. It accepts route configs and a list of "features" produced by functions like `withComponentInputBinding()`, `withPreloading()`, `withDebugTracing()`, `withRouterConfig()`, `withInMemoryScrolling()`, and `withViewTransitions()`. This feature-based composition is a deliberate design so tree-shaking can drop code for features you never opt into.

2.2 Route Configuration (Routes)

A `Route` object is a plain configuration record:

```
interface Route {
  path?: string;
  pathMatch?: 'full' | 'prefix';
  component?: Type<any>;           // or `loadComponent` for lazy standalone components
  redirectTo?: string;
  children?: Routes;
  loadChildren?: () => Promise<any>;
  outlet?: string;
  canActivate?: any[];
  canActivateChild?: any[];
  canDeactivate?: any[];
  canMatch?: any[];
  resolve?: { [key: string]: any };
  data?: Data;
  runGuardsAndResolvers?: RunGuardsAndResolvers;
  title?: string | Type<TitleStrategy>;
}
```

Key fields:

- `path` — the URL segment(s) to match, relative to the parent route. No leading slash.
- `pathMatch` — `'prefix'` (default) matches if the URL *starts with* the path; `'full'` requires the *entire remaining URL* to match. Critical for empty-path redirects (see Gotchas).
- `component` / `loadComponent` — the component to instantiate. `loadComponent` returns a dynamic `import()` for lazy-loaded standalone components.
- `children` — nested route definitions, matched against the remainder of the URL after the parent's path is consumed.
- `loadChildren` — lazy-loads an entire routing configuration (a module or an array of standalone routes) as a separate chunk.
- `outlet` — names the `<router-outlet>` this route targets; omitted means the primary/unnamed outlet.
- `data` — arbitrary static data attached to the route, read via `ActivatedRoute.data`.
- `resolve` — a map of resolver tokens; the router waits for these to resolve before activating the route, then exposes results via `ActivatedRoute.data`.

2.3 RouterOutlet

`<router-outlet>` is a directive that acts as a placeholder marking where the router should render the matched component. It's not a component itself — it inserts the activated component as a sibling in the DOM, immediately after the `<router-outlet>` element.

```
<router-outlet></router-outlet>
```

It exposes lifecycle events (`activate`, `deactivate`, `attach`, `detach`) and can be named:

```
<router-outlet name="sidebar"></router-outlet>
```

Only one **primary** (unnamed) outlet can exist per template level, but multiple named outlets can co-exist at the same level, each independently addressable.

2.4 RouterLink & RouterLinkActive

`routerLink` replaces manual `href` construction and manual click handling with declarative, router-aware navigation that respects the configured `Location` strategy (path vs. hash) and prevents full page reloads.

```
<a routerLink="/products">Products</a>
<a [routerLink]="['/products', product.id]">{{ product.name }}</a>
<a [routerLink]="['/products']" [queryParams]="{ sort: 'asc' }"
  [fragment]="'top'">Sorted</a>
```

`routerLinkActive` adds a CSS class when the bound link's route is currently active — commonly used for nav highlighting:

```
<a routerLink="/home" routerLinkActive="active" [routerLinkActiveOptions]="{exact:
true}">Home</a>
```

`exact: true` requires the *entire* URL to match, not just a prefix (otherwise `/home` would stay "active" while on `/home/details`).

2.5 Route Parameters

Three distinct parameter mechanisms exist, each with different semantics and use cases:

Path (positional) parameters — part of the route's path segment, identify a resource:

```
{ path: 'products/:id', component: ProductDetailComponent }
```

Read via `ActivatedRoute.paramMap` (or the legacy `.params` Observable, or `snapshot.paramMap.get('id')`).

Query parameters — appended after `?key=value&...`, orthogonal to the matched route, used for filters/sorting/pagination that shouldn't affect *which* component loads:

```
/products?category=shoes&page=2
```

Read via `ActivatedRoute.queryParamMap`. Query params persist across navigation unless explicitly cleared (`queryParamsHandling: ''`), and can be merged (`queryParamsHandling: 'merge'`) or preserved (`'preserve'`).

Matrix parameters — key/value pairs scoped to a specific path *segment*, delimited by semicolons, useful when a param applies only to one segment among several (an idea from URI RFC 3986, rarely used elsewhere but native to Angular):

```
/products;category=shoes/details;color=red
```

Read via `paramMap` on the corresponding `ActivatedRoute` segment (matrix params attach to a URL segment, not the whole URL, so nested routes each get their own matrix params). Set them via array syntax in `routerLink`:

```
<a [routerLink]="['/products', { category: 'shoes' }]">Shoes</a>
```

Interview distinction to nail: query params are global to the URL and shared by all activated routes; matrix params are local to a URL segment and can differ between parent and child routes at the same URL.

2.6 Nested (Child) Routes

Child routes let a parent route own a sub-router-outlet, so navigating within a section keeps the parent's shell (nav bars, tabs) mounted while only the child view swaps:

```
{
  path: 'products',
  component: ProductsShellComponent,    // must contain <router-outlet>
  children: [
    { path: '', component: ProductListComponent },
    { path: ':id', component: ProductDetailComponent }
  ]
}
```

The full URL `/products/42` is matched by combining the parent's `products` segment with the child's `:id` segment. Each level of nesting requires its own `<router-outlet>` in the parent component's template.

2.7 Named Outlets (Auxiliary Routes)

Named outlets let multiple independent components render simultaneously in different regions of the same view, each with its own navigable state — e.g., a modal, a chat panel, a sidebar.

```
const routes: Routes = [
  { path: 'inbox', component: InboxComponent },
  { path: 'compose', component: ComposeComponent, outlet: 'popup' }
];
```

```
<router-outlet></router-outlet>
<router-outlet name="popup"></router-outlet>
```

Navigating to an auxiliary route uses object-based link params:

```
<a [routerLink]="[{ outlets: { popup: ['compose'] } }]">New Message</a>
```

The resulting URL shows both outlets' state in parentheses syntax: `/inbox(popup:compose)`. Clearing an outlet: `{ outlets: { popup: null } }`.

2.8 Route Data

Static metadata attached at config time:

```
{ path: 'admin', component: AdminComponent, data: { roles: ['admin'], title: 'Admin Panel' } }
```

Read in the component via `route.snapshot.data['roles']` or reactively via `route.data`. Commonly combined with a `TitleStrategy` to set `document.title`, or with guards that inspect `route.data` to make authorization decisions generically (one guard, parameterized by data, instead of many near-duplicate guards).

Resolved data (from `resolve`) is merged into the same `data` object, keyed by the resolver's token name — this is why resolver keys and static `data` keys share one namespace and can collide if named carelessly.

2.9 Navigation Methods

Imperative navigation (used in component classes, guards, effects):

```
constructor(private router: Router, private route: ActivatedRoute) {}

goToProduct(id: number) {
  this.router.navigate(['/products', id]); // relative to app root by default
  this.router.navigate(['detail', id], { relativeTo: this.route }); // relative navigation
  this.router.navigateByUrl('/products/42?ref=email'); // takes a full URL
  string/UrlTree
}
```

Differences:

- `navigate(commands, extras)` builds a URL from an array of link-parameter segments (like `routerLink`), resolved relative to `extras.relativeTo` (defaults to root if omitted — a common gotcha inside deeply nested components).
- `navigateByUrl(url, extras)` takes an absolute URL string or a `UrlTree` (e.g. produced by `router.createUrlTree(...)` or captured from `router.parseUrl(...)`), bypassing relative-segment resolution.

`NavigationExtras` options worth knowing: `replaceUrl` (replace history entry instead of pushing), `skipLocationChange` (navigate without touching browser history/URL — useful for guard-driven redirects that shouldn't be visible), `state` (attach arbitrary non-serialized data to `history.state`, retrievable via `Router.getCurrentNavigation().extras.state` or `history.state`).

2.10 Router Events

`Router.events` is an `Observable<Event>` emitting a well-defined sequence for every navigation attempt:

1. `NavigationStart`
2. `RouteConfigLoadStart` / `RouteConfigLoadEnd` (only if lazy `loadChildren` / `loadComponent` involved)
3. `RoutesRecognized`
4. `GuardsCheckStart` → `ChildActivationStart` / `ActivationStart` → `GuardsCheckEnd`
5. `ResolveStart` → `ResolveEnd`
6. `ActivationEnd` / `ChildActivationEnd`
7. `NavigationEnd` (success) **or** `NavigationCancel` (guard returned false/`UrlTree`, or a newer navigation superseded this one) **or** `NavigationError` (an exception, e.g. lazy chunk failed to load)
8. `NavigationSkipped` (Angular 15.1+, when navigation is a no-op due to `onSameUrlNavigation` settings)

```
router.events.pipe(
  filter((e): e is NavigationEnd => e instanceof NavigationEnd)
).subscribe(e => console.log('Navigated to', e.urlAfterRedirects));
```

Common use: a top-level loading indicator toggled between `NavigationStart` and `NavigationEnd` | `NavigationCancel` | `NavigationError`.

2.11 Preloading Strategies

Preloading downloads lazy-loaded feature chunks *after* the initial app bootstrap completes, trading a small amount of idle-time bandwidth for near-instant subsequent navigation.

```
provideRouter(routes, withPreloading(PreloadAllModules));
```

- `NoPreloading` (default) — lazy chunks load strictly on-demand, i.e., only when the user actually navigates to a lazy route.
- `PreloadAllModules` — preloads every lazy-loaded route's chunk immediately after bootstrap, in the background, regardless of whether the user is likely to visit it.
- **Custom strategy** — implement `PreloadingStrategy` with a `preload(route, load)` method, letting you preload selectively (e.g., only routes flagged `data: { preload: true }`, or based on network conditions via `navigator.connection`):

```
@Injectable({ providedIn: 'root' })
export class SelectivePreloadingStrategy implements PreloadingStrategy {
  preload(route: Route, load: () => Observable<any>): Observable<any> {
    return route.data?.['preload'] ? load() : of(null);
  }
}
```

2.12 Wildcard Routes

`path: '**'` matches any URL not matched by earlier routes — used for 404/"not found" pages. It must be the **last** entry in the routes array (the router matches top-to-bottom, first match wins), and it cannot have a `path` prefix or `pathMatch` (it inherently matches everything remaining).

```
{ path: '**', component: NotFoundComponent }
```

2.13 Redirect Routes

`redirectTo` rewrites the matched URL to a different path before continuing the matching/activation process.

```
{ path: '', redirectTo: '/dashboard', pathMatch: 'full' },  
{ path: 'old-path', redirectTo: 'new-path' }
```

`pathMatch: 'full'` is mandatory on empty-path redirects — without it, `''` (prefix mode) matches *every* URL (empty string is a prefix of everything), causing the app to always redirect regardless of the actual path. Angular 16+ also supports a function form, `redirectTo: (params) => string`, letting redirects be computed dynamically from matched params/query params.

3. Code Examples

```
// app.routes.ts – standalone route configuration  
import { Routes } from '@angular/router';  
import { authGuard } from '../guards/auth.guard';  
import { productResolver } from '../resolvers/product.resolver';  
  
export const routes: Routes = [  
  { path: '', redirectTo: '/dashboard', pathMatch: 'full' },  
  
  {  
    path: 'dashboard',  
    loadComponent: () =>  
      import('../dashboard/dashboard.component').then(m => m.DashboardComponent),  
    title: 'Dashboard'  
  },  
  
  // Nested routes with a shell component owning a child <router-outlet>  
  {  
    path: 'products',  
    loadComponent: () =>  
      import('../products/products-shell.component').then(m => m.ProductsShellComponent),  
    canActivate: [authGuard],  
    children: [  
      {  
        path: '',  
        loadComponent: () =>  
          import('../products/product-list.component').then(m => m.ProductListComponent)  
      },  
      {  

```

```

        path: ':id',
        loadComponent: () =>
            import('./products/product-detail.component').then(m => m.ProductDetailComponent),
        resolve: { product: productResolver },
        data: { animation: 'detail' }
    }
]
},

// Named (auxiliary) outlet route
{
    path: 'compose',
    outlet: 'popup',
    loadComponent: () =>
        import('./messages/compose.component').then(m => m.ComposeComponent)
},

// Lazy-loaded feature area with its own child routes file
{
    path: 'admin',
    loadChildren: () => import('./admin/admin.routes').then(m => m.ADMIN_ROUTES)
},

{ path: '**', loadComponent: () => import('./not-found/not-found.component').then(m =>
m.NotFoundComponent) }
];

```

```

// app.config.ts – provideRouter with features
import { ApplicationConfig } from '@angular/core';
import { provideRouter, withComponentInputBinding, withPreloading, withViewTransitions,
PreloadAllModules } from '@angular/router';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
    providers: [
        provideRouter(
            routes,
            withComponentInputBinding(), // binds route params/data directly to @Input()
properties
            withPreloading(PreloadAllModules),
            withViewTransitions()
        )
    ]
};

```

```

// main.ts
import { bootstrapApplication } from '@angular/platform-browser';
import { AppComponent } from './app/app.component';
import { appConfig } from './app/app.config';

bootstrapApplication(AppComponent, appConfig);

```



```
// products-shell.component.ts – parent hosting a nested outlet + named outlet sibling
import { Component } from '@angular/core';
import { RouterOutlet, RouterLink, RouterLinkActive } from '@angular/router';

@Component({
  selector: 'app-products-shell',
  standalone: true,
  imports: [RouterOutlet, RouterLink, RouterLinkActive],
  template: `
    <nav>
      <a routerLink="." routerLinkActive="active" [routerLinkActiveOptions]="{exact:
true}">All</a>
      <a [routerLink]="['42']">Product 42</a>
    </nav>
    <router-outlet></router-outlet>      <!-- primary child outlet -->
  `
})
export class ProductShellComponent {}
```

```
// product-detail.component.ts – with withComponentInputBinding(), route params bind
straight to @Input
import { Component, Input } from '@angular/core';

@Component({
  selector: 'app-product-detail',
  standalone: true,
  template: `<h2>Product {{ id }}</h2>`
})
export class ProductDetailComponent {
  @Input() id!: string;      // bound automatically from :id path param
  @Input() product?: any;    // bound automatically from the `product` resolver's result
}
```

```
// auth.guard.ts – functional guard (Angular 15+)
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { AuthService } from './auth.service';

export const authGuard: CanActivateFn = (route, state) => {
  const auth = inject(AuthService);
  const router = inject(Router);
  if (auth.isLoggedIn()) return true;
  return router.createUrlTree(['login'], { queryParams: { returnUrl: state.url } });
};
```

```
// product.resolver.ts – functional resolver
import { inject } from '@angular/core';
import { ResolveFn } from '@angular/router';
import { ProductService } from '../product.service';

export const productResolver: ResolveFn<Product> = (route) => {
  const service = inject(ProductService);
  return service.getById(route.paramMap.get('id')!);
};
```

4. Internal Working

- 1. URL → `UrlTree` parsing.** Every navigation (via link click, `navigate()`, `navigateByUrl()`, or a popstate event) starts by parsing the target URL string into a `UrlTree` — a structured representation with `UrlSegmentGroup`s (segments, children, and per-segment matrix parameters) and top-level query params/fragment. This mirrors the router *config* tree shape, which is what makes matching tractable.
- 2. Recognition (matching).** The `Router` walks the `Routes` config array **top to bottom, depth-first**, trying to consume `UrlSegment`s at each level against each `Route`'s `path`. The first route whose path matches (respecting `pathMatch`) wins at that level — subsequent routes in the array are not tried once a match is found, which is precisely why the wildcard route must be last and why more specific paths must precede more general ones. For each matched route, the router recurses into `children` (or lazy-loads `loadChildren` synchronously-awaited via the observable chain) to consume the remaining segments, building a tree of `ActivatedRouteSnapshot` nodes that mirrors both the config tree and the URL segment tree. This produces `RouterStateSnapshot`, and the `RoutesRecognized` event fires once this is complete.
- 3. Guards phase.** With the new route tree recognized, the router diffs it against the *currently active* route tree to determine which nodes are being: reused unchanged, reused with updated params, deactivated, or newly activated. It then runs, in order: `canDeactivate` (on routes being torn down, deepest first), `canActivateChild` (on ancestors of newly activated nodes), and `canActivate` (on newly activated nodes themselves). Any guard returning `false`, throwing, or returning a redirect `UrlTree` immediately cancels/redirects the navigation (`NavigationCancel`), short-circuiting resolvers and activation entirely.
- 4. Resolve phase.** For nodes surviving guards, any configured `resolve` functions run (in parallel, by default) and the router waits for their observables/promises to complete before proceeding — this is why resolvers are the standard place to prefetch data with zero flash-of-empty-content.
- 5. Router state commit & activation.** The router builds the final `RouterState` (the live, non-snapshot version backed by `ActivatedRoute` objects whose `.params`, `.data`, etc. are Observables rather than static snapshots) and hands it to the `RouterOutlet` directives to render. Each `RouterOutlet`, on `ngOnInit`, subscribed to the router's internal activation stream, calls `viewContainerRef.createComponent()` for newly activated components and destroys the `ComponentRef` for deactivated ones. Because outlets are nested inside the components they instantiate, this activation cascades top-down: a parent's outlet creates the parent-level component, whose template contains the child outlet, which then creates the child-level component — this is why `ngOnInit` fires parent-before-child on initial navigation.
- 6. Reuse vs. recreate.** If a node is matched to the *same* route config both before and after navigation (e.g., `/products/1` → `/products/2`), Angular's default `RouteReuseStrategy` reuses the existing component instance rather than destroying/recreating it — only the `ActivatedRoute`'s `params/data` Observables emit new values. This is the classic gotcha: navigating between sibling routes sharing a component does **not** re-run

`ngOnInit`; you must subscribe to `route.paramMap` (or use `withComponentInputBinding()`) rather than reading params once in `ngOnInit`. Custom `RouteReuseStrategy` implementations can override this (e.g., to cache and restore whole component trees, powering "keep tab state" UX).

7. Change detection & URL commit. After activation, the router updates the browser's `Location` (pushing/replacing history state per `Location` strategy — path-based via the History API, or hash-based via `Location.hash`), then fires `NavigationEnd`. Because component creation/destruction happens inside Angular's execution context, the surrounding `ApplicationRef` change detection pass picks up the new component tree automatically — no manual `markForCheck()` is needed for the router's own activation, though newly created `onPush` components still won't re-render on subsequent unrelated data changes without their own local triggers.

5. Edge Cases & Gotchas

- **Missing `pathMatch: 'full'` on empty-path redirects.** `{ path: '', redirectTo: '/x' }` without `pathMatch: 'full'` matches every URL (prefix mode treats `''` as a prefix of anything) and creates an infinite/always-on redirect loop or an app that can never navigate away from `/x`.
- **Route order matters — first match wins.** A parameterized route (`:id`) placed before a more specific literal route (`new`) will swallow `/products/new`, treating `"new"` as an `:id` value instead of routing to a dedicated "create" component. Always order specific/static paths before dynamic/wildcard ones.
- **Component reuse suppresses `ngOnInit` on param-only changes.** Navigating `/user/1` → `/user/2` when both hit the same route config reuses the component instance. Code that reads `route.snapshot.paramMap` once in `ngOnInit` will show stale data; subscribe to `route.paramMap` (Observable) instead, or rely on `withComponentInputBinding()` so `@Input()` bindings update reactively.
- **`relativeTo` defaults surprise nested components.** `router.navigate(['detail'])` called from deep inside a component tree without `{ relativeTo: this.route }` resolves relative to the `root`, not the component's own `ActivatedRoute`, silently producing the wrong URL.
- **Guards returning a `UrlTree` vs. `false`.** Modern functional guards should return a `UrlTree` (via `router.createUrlTree(...)`) for redirects rather than manually calling `router.navigate()` and returning `false` — doing both can trigger a duplicate/racing navigation. Returning a `UrlTree` is treated by the router as "redirect to this instead," atomically.
- **Matrix params vs. query params confusion.** `;id=1` in a path segment is invisible to `ActivatedRoute.queryParamMap` — it only shows up in that segment's `paramMap`. Mixing them up is a common source of "why is my param undefined" bugs.
- **Lazy `loadChildren` chunk failures produce `NavigationError`, not an exception you can `try/catch` around `navigate()`** — `router.navigate()` returns a Promise that resolves to `false` on failure/cancellation, not a rejected Promise on guard-based cancellation (but a genuinely thrown error, e.g. chunk 404, does reject it). You must distinguish `NavigationCancel` (expected, e.g. guard rejection) from `NavigationError` (unexpected, e.g. network failure) via `router.events`.
- **Auxiliary (named) outlet routes contribute to the URL** in parenthesized syntax (`/inbox(popup:compose)`) — this is intentional (deep-linkable), but easy to be surprised by when debugging "why does my URL look like that."
- **`skipLocationChange` and guard-driven silent redirects** — using this to redirect (e.g.,

unauthenticated users to `/login`) without changing the visible URL means the browser back button behaves unexpectedly, since no history entry was pushed for the "real" destination.

- `RouterModule.forRoot()` **called twice** (e.g., accidentally imported into a lazy feature module instead of `forChild()`) creates a second `Router` instance in some Angular versions, causing duplicated navigation event handling or broken outlet behavior. Standalone `provideRouter()` avoids this class of bug since it's typically supplied once at bootstrap, but multiple calls to `provideRouter` in nested injectors are equally hazardous.
- `PreloadAllModules` **on large apps with many lazy chunks** effectively defeats a large part of the lazy-loading bandwidth benefit at first-load-adjacent time, since everything downloads shortly after bootstrap anyway — a custom selective strategy is usually the production-grade choice.
- **title route property vs. imperative Title service** — mixing both means whichever runs last (`TitleStrategy` default runs on `NavigationEnd`) wins; imperative `titleService.setTitle()` calls inside a component's `ngOnInit` can be clobbered by the router's own title-setting if ordering is wrong.

6. Interview Questions & Answers

Q1. What is the difference between `RouterModule.forRoot()` and `forChild()`?

`forRoot()` configures the router at the application root: it provides the singleton `Router` service, `Location` strategy, and initial route config, and must be called exactly once. `forChild()` registers additional routes for a feature module without re-providing router-level services — calling `forRoot()` in a feature/lazy module accidentally creates a second router configuration and is a classic bug source. The standalone equivalent is calling `provideRouter()` once in `bootstrapApplication / ApplicationConfig`, with feature route arrays simply merged into (or lazy-loaded within) the top-level `Routes` array.

Q2. Explain the difference between path parameters, query parameters, and matrix parameters.

Path parameters (`:id`) are positional, part of a route's URL segment, and identify *which resource* the route matches — e.g. `/products/42`. Query parameters (`?sort=asc`) are global to the entire URL, orthogonal to route matching, and typically represent optional view state like filters/pagination; they persist across navigations if you opt in via `queryParamsHandling`. Matrix parameters (`;color=red`) are scoped to a single URL segment and travel with that segment through nested routing — two sibling segments at different route depths can each carry their own independent matrix params, which query params cannot do since they belong to the whole URL, not a segment.

Interviewer intent: This checks whether the candidate has used matrix params at all (many haven't) and understands *why* Angular has three parameter mechanisms instead of one — it signals depth beyond copy-pasted `routerLink` snippets.

Q3. Why must `path: '**'` (wildcard route) be the last entry in the routes array?

The router matches routes top-to-bottom and stops at the first match. A wildcard matches any remaining URL unconditionally, so if placed earlier it would swallow every subsequent route below it, making them unreachable. It also cannot specify `pathMatch`, since matching "everything" makes prefix/full distinction moot.

Q4. What's the purpose of `pathMatch: 'full'`, and what breaks if you omit it on an empty-path redirect?

`pathMatch` controls whether a route's `path` needs to match the *entire* remaining URL (`'full'`) or merely be a *prefix* of it (`'prefix'`, the default). For `{ path: '', redirectTo: '/dashboard' }`, an empty string is technically a prefix of every URL, so in default `'prefix'` mode this redirect fires unconditionally regardless of the actual path, breaking navigation to any other route. Adding `pathMatch: 'full'` restricts the redirect to only when the remaining URL is *exactly* empty (i.e., truly at the app root).

Q5. How do nested routes and their `<router-outlet>` relate to each other?

Each level of route nesting requires the parent-matched component to contain its own `<router-outlet>` to host its children's activated components. The URL is matched by concatenating path segments across levels (`products/:id` = parent `products` + child `:id`), and the `ActivatedRoute` tree mirrors this nesting — each level has its own `ActivatedRoute` with its own params/data, accessible via `route.parent` / `route.children` or injected directly into the corresponding component.

Q6. What are named (auxiliary) outlets used for, and how do you navigate to one?

Named outlets let independent components be activated and navigated in parallel regions of the same view — e.g., a chat popup alongside a main inbox view — each contributing its own segment to the URL in parenthesized syntax (`/inbox(popup:compose)`). You declare them with `outlet: 'popup'` in the route config and a matching `<router-outlet name="popup">` in the template, then navigate via the object-based link-params array: `router.navigate([{ outlets: { popup: ['compose'] } }])`. Clearing one sets its outlet to `null`.

Q7. What's the difference between `router.navigate()` and `router.navigateByUrl()`?

`navigate(commands, extras)` builds a `UrlTree` from an array of link-parameter segments, resolved relative to `extras.relativeTo` (root by default) — essentially the imperative equivalent of `[routerLink]="[...]"`. `navigateByUrl(url, extras)` accepts a complete, already-resolved URL string or `UrlTree` and navigates to it directly, without any relative-path resolution logic — useful when you already have a full target URL (e.g., a `UrlTree` built via `router.createUrlTree()` inside a guard, or a URL retrieved from server data).

Q8. Walk through the sequence of `Router.events` fired during a typical navigation, including a guard rejection.

`NavigationStart` → (if lazy: `RouteConfigLoadStart` / `RouteConfigLoadEnd`) → `RoutesRecognized` → `GuardsCheckStart` → per-node `ChildActivationStart` / `ActivationStart` as guards run → `GuardsCheckEnd`. If any guard returns `false` or fails, the sequence terminates immediately with `NavigationCancel` — `ResolveStart` / `ResolveEnd` and activation events never fire. On success it continues: `ResolveStart` → `ResolveEnd` → `ActivationStart` / `ActivationEnd` per node → `NavigationEnd`. An unhandled exception (e.g., a lazy chunk 404) produces `NavigationError` instead of `NavigationEnd`.

Interviewer intent: Tests whether the candidate has actually debugged navigation issues via `Router.events` (e.g., building a loading spinner or investigating a "navigation silently does nothing" bug) rather than only reading about it.

Q9. What is a `RouteReuseStrategy`, and why does navigating between two URLs matching the same route config not re-run `ngOnInit`?

When Angular recognizes the new route tree, it diffs it against the currently active tree per node. If a node matches the *same* `Route` config before and after (differing only in params, e.g. `/user/1` → `/user/2`), Angular's default `RouteReuseStrategy` reuses the existing `ActivatedRoute` / component instance rather than destroying and recreating it, updating only the `params` / `data` Observables. `ngOnInit` therefore only fires once across such navigations — components must subscribe to `route.paramMap` (or use `withComponentInputBinding()`) to react to subsequent param changes. Implementing a custom `RouteReuseStrategy` lets you change this behavior entirely, e.g., caching detached component trees to restore state when navigating back to a previously-visited route (common in tabbed UIs).

Q10. How do resolvers interact with guards and route `data` in terms of timing?

Guards (`canActivate` / `canActivateChild` / `canDeactivate`) run first; any rejection or redirect short-circuits the navigation before resolvers ever execute. Only once all applicable guards pass does the router run configured `resolve` functions (in parallel by default), and it holds navigation — the outlet does not activate

the new component, and the URL does not update — until all resolvers complete. Resolved values are merged into the same `ActivatedRoute.data` object as any static `data` configured on the route, keyed by the resolver's property name, so a resolver key colliding with a static `data` key will silently overwrite one or the other depending on merge order.

Q11. What's the difference between `NoPreloading`, `PreloadAllModules`, and a custom `PreloadingStrategy`?

`NoPreloading` (implicit default) loads lazy `LoadChildren / LoadComponent` chunks strictly on first navigation to them — smallest possible initial bundle but a network round-trip delay on first visit to each lazy area.

`PreloadAllModules` preloads every lazy chunk in the background immediately after the initial route activates, trading idle bandwidth for near-instant later navigation, but can waste bandwidth on rarely-visited areas and slightly compete with initial-render work on constrained connections. A custom strategy implements `PreloadingStrategy.preload(route, load)`, letting you decide per-route (e.g., via `route.data.preload` flags, user role, or `navigator.connection.effectiveType`) whether to call `load()` — the standard production answer when an app has many lazy routes of varying importance.

Q12. Why does Angular require `withComponentInputBinding()` (or manual subscription to `ActivatedRoute`) instead of just injecting params as plain properties automatically?

Route params, query params, and resolved data are inherently *dynamic* — they can change without the component being destroyed and recreated (see route reuse, Q9) — so they're modeled as Observables (`ActivatedRoute.params`, `.queryParams`, `.data`) reflecting live navigation state rather than one-time constructor injection values. `withComponentInputBinding()` (Angular 16+) is sugar that automatically subscribes on the component's behalf and pushes values into matching `@Input()` properties via `SimpleChanges`, giving you the ergonomics of plain inputs while preserving reactivity — but it's opt-in specifically because it changes change-detection-triggering behavior (each param/data emission now flows through Angular's normal input-binding change detection path) and because before Angular 16 there was no mechanism to project router state into inputs at all.

Interviewer intent: Distinguishes candidates who understand *why* Angular models router data reactively (architecture) from those who only know the syntax (`@Input() id`) without knowing what's happening underneath, or the version/opt-in nuance.

Q13. How does the router determine which components to destroy and which to keep when navigating from `/a/x` to `/a/y` where both share a common ancestor route?

The router recognizes the new URL into a full route tree, then walks both the old and new `ActivatedRouteSnapshot` trees together, node by node, from the root down. For each position, if the matched `Route` config object is identical and (per the `RouteReuseStrategy`) reusable, the existing component/injector subtree is kept and only its params/data update; the moment a node differs (different route matched, or the reuse strategy says no), that node and *everything below it* is torn down (`canDeactivate` already having been checked earlier) and freshly activated. So shared ancestors (e.g., a shell route both `x` and `y` sit under) are untouched, while the diverging leaf component is destroyed and recreated — unless it happens to resolve to the same route config, in which case Q9's reuse behavior applies instead.

Q14. What happens if a `canActivate` guard on a lazy-loaded route needs to prevent the chunk itself from downloading (not just prevent activation after it's loaded)?

Standard `canActivate` runs *after* recognition, which for a `LoadChildren / LoadComponent` route means the chunk has already been fetched (recognition needs the child routes to know what's inside). To prevent even the network request/download for an unauthorized user, use `canMatch` instead — it runs during the route *matching* phase, before the router commits to that route (and before triggering its lazy import), and can cause the router to fall through to try other sibling routes (e.g., a different lazy chunk, or a "forbidden" route) instead.

This is the correct guard type for scenarios like "don't even let unauthenticated users' browsers download the admin module's JS."

7. Quick Revision Cheat Sheet

- **Setup:** `provideRouter(routes, ...features)` (standalone) or `RouterModule.forRoot() / forChild()` (NgModule). `forRoot` once at root only.
- **Route fields:** `path`, `pathMatch` ('full' vs 'prefix'), `component / loadComponent`, `children`, `loadChildren`, `outlet`, `redirectTo`, `data`, `resolve`, `canActivate / canMatch / canDeactivate`.
- **Matching:** top-to-bottom, first match wins, depth-first — order specific paths before dynamic ones, wildcard `**` always last.
- `<router-outlet>`: placeholder for activated component; unnamed (primary) + named outlets can coexist; nested routes need one outlet per nesting level.
- `routerLink`: array syntax for path segments; `[queryParams]`, `[fragment]`; `routerLinkActive` + `routerLinkActiveOptions.exact` for nav highlighting.
- **Params:** path params (`:id`, positional, per-segment) vs. query params (`?k=v`, whole-URL, persistable via `queryParamsHandling`) vs. matrix params (`;k=v`, per-segment, sibling-independent).
- **Nested routes:** parent path + child path concatenate; parent component needs its own `<router-outlet>`.
- **Named outlets:** `outlet: 'name'` in config; URL syntax `/path(name:childPath)`; navigate via `[[ outlets: { name: [...] } ]]`.
- **Route data:** static `data` + resolved `resolve` results merge into one `ActivatedRoute.data` object.
- **Navigation:** `navigate(commands, { relativeTo, queryParams, replaceUrl, skipLocationChange, state })` vs. `navigateByUrl(urlOrTree)`.
- **Events order (success):** `NavigationStart` → `RoutesRecognized` → `GuardsCheckStart/End` → `ResolveStart/End` → `ActivationEnd` → `NavigationEnd`. Failure paths: `NavigationCancel` (guard/redirect) or `NavigationError` (exception).
- **Preloading:** `NoPreloading` (default, on-demand) vs `PreloadAllModules` (eager background) vs custom `PreloadingStrategy` (selective, via `route.data`).
- **Wildcard:** `{ path: '**', component: NotFoundComponent }`, must be last.
- **Redirect:** `{ path: '', redirectTo: '/x', pathMatch: 'full' }` — never forget `pathMatch: 'full'` on empty-path redirects.
- **Reuse:** same route config across navigation → component instance reused, `ngOnInit` doesn't refire → subscribe to `paramMap / data` or use `withComponentInputBinding()`.
- **Guard timing:** `canMatch` (before chunk load/route commit) < `canActivate / canActivateChild` (after recognition, before resolve) < `resolve` (before activation) < `canDeactivate` (checked on outgoing routes before any of the above for the incoming route).

Created By - Durgesh Singh